Московский государственный технический университет им. Н.Э. Баумана

Факультет ИУ
Кафедра ИУ5

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №2

Вариант Б10

Выполнил:
Студент группы ИУ5-32Б:
Овчинников И. В .
Подпись и дата:

Проверил:
Преподаватель каф. ИУ5
Гапанюк Ю.Е.
Подпись и дата:

Москва, 2025

Текст программы

Task/main.py

```
from models import Browser, Computer, BrowserComputer
from queries import *

def main():
    computers = [
        Computer(1, "Игровой ПК 'Омега'"),
        Computer(2, "Ноутбук 'Lenovo IdeaPad'"),
        Computer(3, "Сервер 'Atlas'"),
        Computer(4, "Рабочая станция 'Delta'"),
    ]

    browsers = [
        Browser(1, "Google Chrome", 120, 1),
        Browser(2, "Mozilla Firefox", 118, 1),
        Browser(3, "Яндекс.Браузер", 23, 2),
        Browser(4, "Opera", 102, 3),
        Browser(5, "Edge", 115, 4),
        Browser(6, "Браузеров", 5, 4),
    ]

    browser_computers = [
        BrowserComputer(1, 1),
        BrowserComputer(1, 2),
        BrowserComputer(2, 3),
        BrowserComputer(3, 4),
        BrowserComputer(4, 5),
        BrowserComputer(4, 6),
        BrowserComputer(1, 3),
        BrowserComputer(2, 1),
    ]

    res1 = query_one_to_many(browsers, computers)
    res2 = query_computers_with_browser_count(res1)
    res3 = query_many_to_many(browsers, computers, browser_computers)

    print(res1)
    print(res2)
    print(res3)

if __name__ == "__main__":
    main()
```

task/models.py

```
class Browser:
    def __init__(self, id, name, version, computer_id):
        self.id = id
        self.name = name
        self.version = version
        self.computer_id = computer_id


class Computer:
    def __init__(self, id, name):
        self.id = id
        self.name = name


class BrowserComputer:
    def __init__(self, computer_id, browser_id):
        self.computer_id = computer_id
        self.browser_id = browser_id
```

task/queries.py

```
def query_one_to_many(browsers, computers):
    return sorted(
        [
            (b.name, b.version, c.name)
            for b in browsers
            for c in computers
            if b.computer_id == c.id
        ],
        key=lambda x: x[0]
    )


def query_computers_with_browser_count(one_to_many):
    result = {}
    for _, _, computer_name in one_to_many:
        result[computer_name] = result.get(computer_name, 0) + 1

    return sorted(result.items(), key=lambda x: x[1])


def query_many_to_many(browsers, computers, browser_computers):
    computer_map = {c.id: c.name for c in computers}
    browser_map = {b.id: b.name for b in browsers}

    pairs = [
```

```
        (browser_map[bc.browser_id], computer_map[bc.computer_id])
        for bc in browser_computers
        if bc.browser_id in browser_map and bc.computer_id in computer_map
    ]

    return sorted(
        [(b, c) for b, c in pairs if b.endswith("ов")],
        key=lambda x: x[0]
    )
```

Tests/conftest.py

```
import pytest

from task.models import Browser, Computer, BrowserComputer


@pytest.fixture
def data():
    computers = [
        Computer(1, "PC1"),
        Computer(2, "PC2"),
    ]

    browsers = [
        Browser(1, "Chrome", 120, 1),
        Browser(2, "Firefox", 118, 1),
        Browser(3, "Браузеров", 5, 2),
    ]

    relations = [
        BrowserComputer(1, 1),
        BrowserComputer(1, 2),
        BrowserComputer(2, 3),
    ]

    return browsers, computers, relations
```

tests/test_query.py

```
import pytest

from task.queries import *
from .conftest import data


def test_query_one_to_many(data):
    browsers, computers, _ = data
```

```
    result = query_one_to_many(browsers, computers)

    assert result == [
        ("Chrome", 120, "PC1"),
        ("Firefox", 118, "PC1"),
        ("Браузеров", 5, "PC2"),
    ]


def test_query_computers_with_browser_count(data):
    browsers, computers, _ = data
    one_to_many = query_one_to_many(browsers, computers)

    result = query_computers_with_browser_count(one_to_many)

    assert result == [
        ("PC2", 1),
        ("PC1", 2),
    ]


def test_query_many_to_many(data):
    browsers, computers, relations = data

    result = query_many_to_many(browsers, computers, relations)

    assert result == [
        ("Браузеров", "PC2"),
    ]
```

Скриншоты работы приложения

```
(venv) deviatovilya@MacBook-Air-cua-Deviatov rk2 % python task/main.py
[('Edge', 115, "Рабочая станция 'Delta'"), ('Google Chrome', 120, "Игровой ПК 'Омега'"), ('Mozilla Firefox', 118
, "Игровой ПК 'Омега'"), ('Opera', 102, "Сервер 'Atlas'"), ('Браузеров', 5, "Рабочая станция 'Delta'"), ('Яндекс
.Браузер', 23, "Ноутбук 'Lenovo IdeaPad'")]
[("Сервер 'Atlas'", 1), ("Ноутбук 'Lenovo IdeaPad'", 1), ("Рабочая станция 'Delta'", 2), ("Игровой ПК 'Омега'",
2)]
[('Браузеров', "Рабочая станция 'Delta'")]
(venv) deviatovilya@MacBook-Air-cua-Deviatov rk2 % pytest .
============================================ test session starts ============================================
platform darwin -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/deviatovilya/Desktop/projects/cslabs-3/rk2
collected 3 items

tests/test_query.py ...                                                                            [100%]

============================================= 3 passed in 0.01s =============================================
(venv) deviatovilya@MacBook-Air-cua-Deviatov rk2 %
```

Рис. 1 – Скриншот работы приложения